Chunked Bump Allocator with Selective Reset

Joanita Nakityo
UNIST
Department of Computer Science and Engineering
Ulsan, South Korea

1 Introduction

Memory allocation performance is critical in systems software, compilers, and runtime environments. Bump allocators are widely used due to their extremely fast allocation—reducing allocation to a simple pointer increment. This design has made them essential in performance-critical systems such as the Rust compiler (rustc), LLVM, game engines, and JavaScript runtimes.

However, traditional bump allocators suffer from a fundamental limitation: all-or-nothing deallocation. Memory can only be reclaimed by resetting the entire region, assuming all allocated objects share the same lifetime. Real-world programs frequently violate this assumption, exhibiting mixed-lifetime workloads where some objects are long-lived while others are short-lived. In such cases, short-lived allocations cannot be reclaimed promptly, causing significant memory waste.

This work addresses this limitation by proposing a *Chunked Bump Allocator with Selective Reset*. The allocator divides memory into multiple independent chunks, each acting as a mini-arena that can be reset independently when empty. This design preserves the performance benefits of bump allocation while enabling fine-grained memory reclamation.

Contributions:

- A chunked bump allocator design supporting selective, per-chunk reset
- A safe Rust implementation leveraging ownership and RAII
- Empirical evaluation demonstrating 60–87% memory savings with predictable performance

2 Background and Related Work

Region-based memory management organizes memory into regions where objects are allocated together and freed in bulk [1]. Bump allocators realize this concept by sequentially advancing a pointer within a region, enabling constant-time allocation with minimal overhead. However, they impose strict lifetime discipline: all objects within a region must be freed simultaneously.

When workloads exhibit mixed lifetimes—where some objects are short-lived while others persist—this all-or-nothing constraint leads to memory inefficiency. Short-lived objects remain resident until the entire region resets, even when no longer needed. This limitation restricts bump allocator applicability and forces developers to carefully structure memory usage.

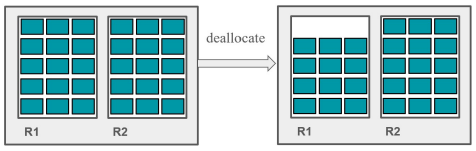


Figure 1. Chunked bump allocator architecture. Memory is divided into independent chunks, each supporting bump allocation and selective reset when empty.

Modern general-purpose allocators such as mimalloc employ memory chunking to improve locality and cache performance [2], but rely on free lists and per-object deallocation rather than region-based allocation. Conversely, traditional Rust bump allocators like bumpalo lack chunking and selective reset capabilities, requiring full region resets.

This work combines chunked organization with selective reset to address this limitation in the Rust ecosystem.

3 Design and Implementation

3.1 Chunked Architecture and Selective Reset

The proposed allocator organizes memory into multiple fixed-size chunks, where each chunk functions as an independent mini-arena with its own allocation pointer and metadata. Allocation requests are satisfied by bump-allocating within an existing chunk with sufficient free space. If no suitable chunk exists, a new chunk is allocated. This preserves the simplicity and speed of traditional bump allocation while enabling fine-grained memory management.

To support selective reclamation, each chunk maintains an atomic reference count tracking the number of live allocations originating from that chunk. When an allocation is created, the chunk's reference count increments. When the allocation is dropped, Rust's RAII mechanism automatically decrements the count via the allocation guard's `Drop` implementation.

A chunk becomes eligible for reset when its reference count reaches zero and it has been used. At this point, the chunk's allocation pointer resets to its initial position, reclaiming all memory. This mechanism allows memory from short-lived allocations to be reclaimed independently, without affecting other chunks containing long-lived objects. Figure 1 illustrates this architecture.

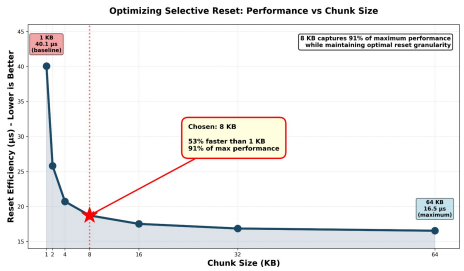


Figure 2. Impact of chunk size on reset performance. Throughput improves dramatically from 1 KB to 8 KB, then flattens, making 8 KB optimal for balancing performance and reset granularity.

3.2 Safety and Chunk Size Selection

The allocator leverages Rust's ownership model to enforce safe usage [3]. Each allocation returns a guard object whose `Drop` implementation automatically updates the originating chunk's reference count. This ensures consistency even during early returns or panic unwinding, maintaining memory safety without manual bookkeeping.

Chunk size critically balances allocation throughput and reset granularity. We evaluated sizes from 1 KB to 64 KB under mixed-lifetime workloads. Figure 2 shows that throughput improves significantly from 1 KB to 8 KB (53% improvement), with diminishing returns beyond 8 KB. Smaller chunks require more frequent chunk creation and reset scanning, while larger chunks reduce reset effectiveness by mixing more allocations per chunk.

Based on these results, 8 KB was selected as optimal, capturing 91% of maximum performance while maintaining sufficient granularity for effective selective reset. This configuration requires approximately 8 chunks for 1,000 allocations of 64 bytes, allowing substantial memory reclamation in mixed-lifetime scenarios.

4 Evaluation

4.1 Experimental Setup

We evaluated the allocator using synthetic workloads with controlled allocation patterns. Each workload consisted of 1,000 allocations of 64 bytes with varying short-lived to long-lived ratios: 90/10, 70/30, and 50/50. Allocations were dropped according to their lifetime category, followed by selective reset.

We compared against two baselines: *bumpalo*, a standard Rust bump allocator, and Rust's *default allocator*, representing general-purpose heap allocation. All experiments were single-threaded to ensure fairness, as both the proposed allocator and bumpalo are single-threaded by default.

Metrics measured include peak memory usage, allocation throughput, and memory efficiency (live data / peak memory).

4.2 Results

Memory Efficiency. The chunked allocator significantly reduced peak memory usage across all workloads. Figure 3a shows that at the 90/10 ratio, peak memory decreased from approximately 64 KB for bumpalo to 8 KB for the proposed allocator—an 8× reduction. Similar reductions of 62% and 49% were observed at 70/30 and 50/50 ratios respectively, demonstrating consistent memory reclamation.

Figure 3b shows memory efficiency across workload patterns. The proposed allocator maintains 75–100% efficiency across all ratios, while bumpalo's efficiency degrades from 10% to 50% as short-lived allocations increase. This degradation occurs because bumpalo cannot reclaim memory until full arena reset, leaving freed short-lived objects occupying space indefinitely. Selective reset effectively eliminates this waste.

Throughput. Figure 4 compares allocation time across lifetime ratios. The chunked allocator incurs approximately 5× throughput overhead relative to bumpalo (19 μs vs. 3.7 μs average) due to reference counting and chunk management. Each allocation and deallocation updates per-chunk metadata to enable selective reset.

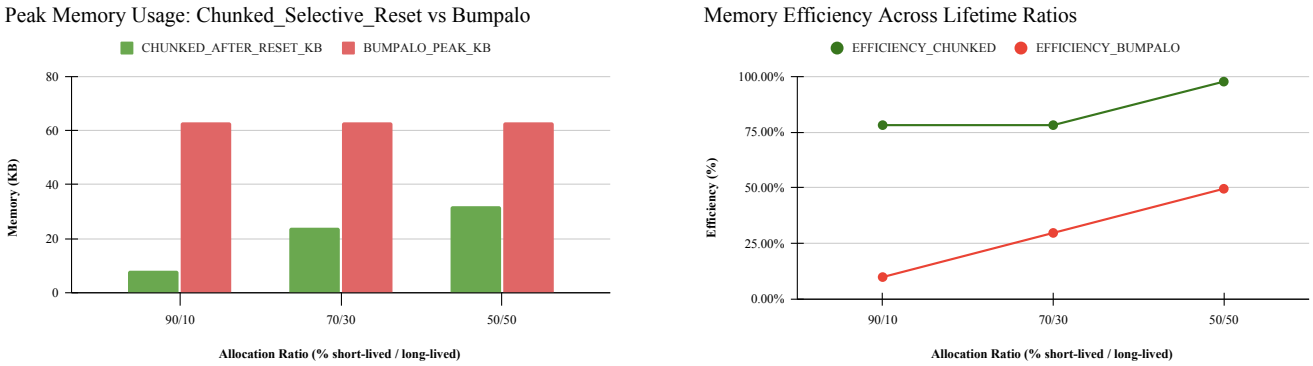
However, the proposed allocator maintains stable, predictable performance across all workload ratios, with less than 15% variation between 90/10 and 50/50 patterns. In contrast, the default allocator exhibits 2.8× performance degradation (6 μs to 23 μs) as long-lived allocations increase, due to growing management overhead and fragmentation.

This stability arises because allocation in the chunked design remains a bounded operation within a chunk, independent of free-list traversal or global allocator state. While bumpalo remains fastest due to minimal pointer-bump allocation, it lacks selective memory reclamation. The proposed allocator trades peak throughput for predictable performance and significantly improved memory efficiency.

5 Discussion

Performance Trade-offs. The allocator explicitly trades allocation throughput for memory efficiency and predictability. While bumpalo achieves the lowest allocation latency through pure pointer-bump allocation, it cannot reclaim memory selectively. The 5× throughput overhead is acceptable in memory-constrained scenarios where avoiding out-of-memory conditions is critical, or where allocation frequency is not the primary bottleneck.

Applicability. This design particularly benefits systems with mixed-lifetime workloads. Multi-pass compilers can free AST memory after IR generation while retaining IR for code generation. Game engines can reset per-frame particle effects while preserving level geometry. Web servers can reclaim request parsing buffers while maintaining session data. For workloads where all allocations share uniform lifetimes, traditional bump allocators remain optimal.



(a) Peak memory usage across allocation ratios

(b) Memory efficiency across allocation ratios

Figure 3. Memory efficiency results. (a) Selective reset reduces peak memory by up to 8× compared to bumpalo. (b) The chunked allocator maintains 75–100% efficiency while bumpalo degrades significantly.

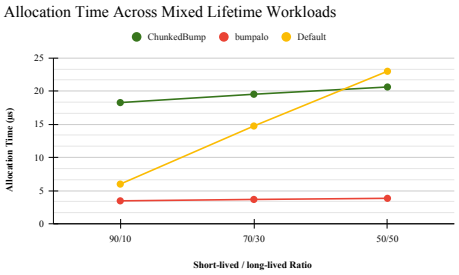


Figure 4. Allocation time under mixed-lifetime workloads. The chunked allocator shows higher overhead than bumpalo but maintains stable performance, unlike the default allocator's 2.8× degradation.

Limitations. The current implementation is single-threaded, consistent with bumpalo and most bump allocators. However, the chunk-based design naturally extends to multi-threading by assigning per-thread chunk pools or using atomic operations for thread-safe chunk management. Chunk size is currently fixed at 8 KB; adaptive sizing based on allocation patterns could further optimize memory-performance tradeoffs. Additionally, very large allocations exceeding chunk size require special handling, which the current implementation does not address.

6 Conclusion and Future Work

This work presented a Chunked Bump Allocator with Selective Reset, addressing the fundamental all-or-nothing deallocation limitation of traditional bump allocators. By dividing memory into independently resettable chunks tracked via reference counting, the allocator enables efficient memory reuse in mixed-lifetime workloads while preserving the simplicity of bump allocation.

Experimental results demonstrate 60–87% reductions in peak memory usage while maintaining predictable performance across varying workload patterns. The explicit tradeoff—5× throughput overhead for 8× memory efficiency—makes this approach particularly suitable for memory-constrained systems, long-running applications, and scenarios where predictable performance is valued over peak throughput.

Future work includes extending the allocator to support multi-threaded environments through per-thread chunk pools, exploring adaptive chunk sizing strategies to dynamically optimize for different allocation patterns, and evaluating performance on real-world applications such as compiler implementations and game engines.

References

[1] Dan Grossman, Greg Morrisett, Trevor Jim, Michael Hicks, Yanling Wang, and James Cheney. 2002. Region-based memory management in cyclone. In *Proceedings of the ACM SIGPLAN 2002 Conference on Programming Language Design and Implementation* (Berlin, Germany) *(PLDI '02)*. Association for Computing Machinery, New York, NY, USA, 282–293. doi:10.1145/512529.512563

[2] Daan Leijen, Benjamin Zorn, and Leonardo Moura. 2019. *Mimalloc: Free List Sharding in Action.* 244–265. doi:10.1007/978-3-030-34175-6_13

[3] Nicholas D. Matsakis and Felix S. Klock. 2014. The rust language. *Ada Lett.* 34, 3 (Oct. 2014), 103–104. doi:10.1145/2692956.2663188